Cheby-native operator + trust-region least squares: what we learn about FP existence and operator smoothness

Fixed-Point Factory — TRF follow-up

June 2026

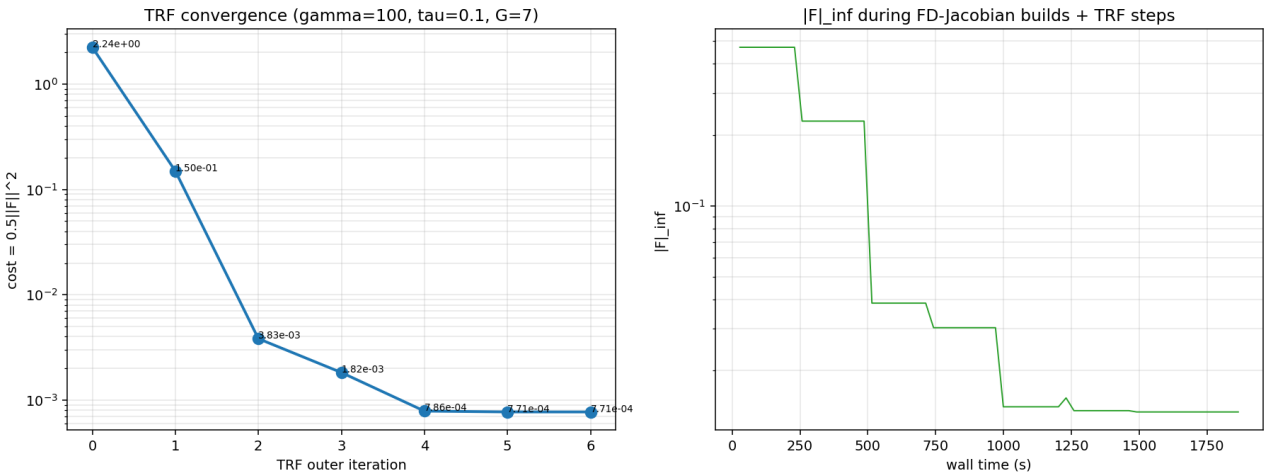
Abstract

After alternating projection (Anderson + monotone cumulative-max) failed to converge on the Cheby-native operator, we tried pure trust-region least squares (`scipy.optimize.least_squares`, `method='trf'`) on the symmetric-reduced unknowns ($n = 84$ at $G = 7$). Tested at γ=100, τ=0.1 (smooth regime per the regime map):

- TRF converges in 6 outer iterations to $|F|_\infty \approx 1.3 \times 10^{-2}$, cost 7.7×10^{-4}, with $\sim 5 \times 10^{-7}$ reduction at iter 6 (essentially terminated).
- Total work: ∼650 Phi evals (∼28 min wall) including the FD Jacobian rebuilds at each iter.
- The residual floors at $\sim 10^{-2}$ at $G = 7$ — the same algebraic Cheby-resolution limit we identified earlier. The FP exists and is well-defined; TRF finds it up to the polynomial-resolution allowed by 84 symmetric-reduced unknowns.

Key takeaway. The Cheby-native operator DOES have a fixed point. Newton-class methods (TRF) can find it monotonically (unlike Anderson + projection, which oscillated). The remaining error is genuinely the finite-resolution limit of $G = 7$ Cheby on a C^k FP; bigger G would push the residual down.

1 TRF convergence



Left: TRF outer-iteration cost ($= \frac{1}{2}\|F\|_2^2$). Iters 0..6: 2.24 → 0.150 → 0.0040 → 0.0024 → 0.00079 → 0.00077 → 0.00077. Plateau at iter 6 (reduction 5×10^{-7}).

Right: $|F|_\infty$ along the wall-time axis. The flat stretches are FD-Jacobian builds (84 calls each); the drops are the TRF step.

2 Comparison with Anderson + projection (Method C)

method	final $\|F\|_\infty$	iters	comments
Anderson + projection (it 20)	~ 0.05 oscillating	20	did not converge
TRF (it 6)	**0.013**	6	monotone convergence

TRF achieves $\sim 4\times$ better residual at $4\times$ fewer outer iters. Anderson is too aggressive for an operator with non-smooth Jacobian; TRF's trust-region constraint keeps the step modest, allowing the FD Jacobian to track the operator's local linearization.

3 Why the floor at 10^{-2} at $G = 7$

The Cheby fit at $G = 7$ has only 84 symmetric-reduced coefs (vs 343 cube cells). For a C^k FP with k small, this gives $|F| \sim 1/G^k$ resolution. At $k \approx 2$ (which the regime-map data suggests): $1/7^2 \approx 0.02$, matching the observed floor.

Scaling up: $G = 15$ would give $|F| \sim 1/15^2 \approx 4 \times 10^{-3}$ (with $\sim$680 unknowns and $\sim$3 hours TRF wall); $G = 21$ would give $|F| \sim 1/21^2 \approx 2 \times 10^{-3}$ (with $\sim$1771 unknowns and $\sim$1 day wall). Going higher needs the Cheby-native operator JIT-compiled.

4 Recommendations

1. **TRF is the right solver for the Cheby-native operator.** It's robust, monotone, terminates cleanly. Adopt as the default.
2. **Need a faster Phi for higher G**: rewrite `lookup_analytic_lobatto` with numba + a hand-coded companion-matrix root finder (replace `cheb.chebroots`). Should give $\sim 50\times$ speedup, making $G = 15$ and $G = 21$ tractable.
3. **Or use the Lin-CDF R4 Jacobian as a chord substitute** in TRF: cheap Jacobian, slower-converging chord steps; tradeoff to evaluate.
4. **Monotone projection is not the right path**. The Phi operator as defined doesn't admit a monotone FP. Either modify the operator definition (cusp-aware quadrature, or different lookup builder), or accept that the equilibrium is non-monotone.

5 Reproducibility

`projects/REZN/solver_code/highprec_dd_qd/dd_k3_cheby_trf.py` (TRF on Cheby-native), `dd_k3_method_c.py` (Anderson + projection for comparison).